

Module 21: Testing Your Code

This curriculum has a zero-dependency rule, and JUnit is a dependency. So this module teaches testing without one, which turns out to be the more useful lesson anyway: a test framework is not magic, and seeing the whole of a working one makes the real ones easier to reason about.

It also covers `assert`, mainly so you know why it is **not** a testing tool.

What You'll Learn

- Why every `assert` in your program does nothing by default
- The two ways `assert` gets misused, one of which is a real production bug
- A complete test harness in 39 lines
- What a framework adds that you cannot reasonably hand-roll

Coming From Another Language

	Python	Java
Built-in test runner	<code>unittest</code> , <code>pytest</code>	none in the JDK
<code>assert</code> in tests	the normal way	disabled by default
Discovery	by naming convention	by annotation, via a framework
Standard choice	<code>pytest</code>	JUnit 5, effectively universal

The row that matters: Python's `assert` is on unless you ask otherwise. Java's is off unless you ask for it, and almost nobody asks.

The Lesson

Assertions are off

From `AssertionsAreOff.java`:

```
without -ea:  assertions enabled: false
with -ea:    assertions enabled: true
```

The detection uses a trick worth recognising:

```
boolean enabled = false;
assert enabled = true;
```

The assignment only runs if assertions are enabled, because the whole statement is skipped otherwise. It is one of very few places where a side effect inside an `assert` is correct.

Two rules follow.

Never put logic inside an assert. This is a real bug:

```
assert list.remove(item);
```

It works in tests with `-ea` and silently stops removing anything in production.

Never use `assert` to validate arguments from callers. Use an exception, which is always on:

```
if (n < 0) throw new IllegalArgumentException("n must not be negative");
```

`assert` is for **internal invariants you believe cannot be false**: an unreachable switch default, a state your own code has just established. [UsingAssertions.java](#) shows both, and uses the message form, which costs one string and turns a null-message `AssertionError` into one that explains itself.

A note on this repository: the verification script runs examples with plain `java <file>`, so assertions are off. [UsingAssertions.java](#) carries an `// EXPECT: assertions` marker so the script runs it with `-ea`. Without that, an example teaching `assert` would pass by doing nothing at all.

A test harness in 39 lines

[MicroHarness.java](#) contains a complete `Tests` class. Its real output:

```
PASS    addition works
PASS    string concatenation
PASS    list size
FAIL    this one is wrong on purpose
        returned false
FAIL    this one throws
        IllegalStateException: boom
PASS    equality with a useful message
FAIL    and one that fails
        expected <5> but was <4>

7 run, 4 passed, 3 failed
```

Three design points worth noticing, because every real framework makes the same ones:

1. **A test that throws is a failure, not a crash of the run.** The harness catches `Throwable`, which is one of the few places that is correct: it must survive anything a test does to it.
2. **Failures carry a detail**, so `expected <5> but was <4>` appears rather than a bare `FAIL`.

3. **A real harness exits non-zero** when anything failed, so CI notices. This one does not, because it has to run clean under this repository's own verification.

What a framework adds

From `WhatJUnitAdds.java`:

1. Discovery. The micro harness needs every test registered by hand. Forget a line and the test silently does not exist, and nothing reports a test that was never registered. JUnit scans for `@Test` and that failure mode disappears.

2. Isolation. Every `@Test` gets a fresh instance, so state cannot leak between tests. Without it:

```
test one added an item; list size is now 1
test two starts with size 1, not 0
```

A test that passes only because a previous one ran is worse than no test, because it fails when someone reorders them.

3. Assertion messages. `assertEquals` on a ten-field object pointing at the differing field is the whole debugging session.

4. The things there is no room to hand-roll: parameterised tests, `assertThrows`, timeouts, tagging, IDE integration, and a standard report format CI can read.

```
class CalculatorTest {
    @Test
    void addsTwoNumbers() {
        assertEquals(4, Calculator.add(2, 2));
    }

    @Test
    void rejectsNegativeInput() {
        assertThrows(IllegalArgumentException.class, () -> Calculator.sqrt(-1));
    }
}
```

On a real project, add JUnit 5 on day one. It is the default everywhere and nothing about it is controversial. What carries over from this module is not the harness: it is knowing that a test is a method that checks something and reports, that isolation is a deliberate feature, and that `assert` is not a testing tool.

Run It

```
java modules/21-testing-your-code/examples/AssertionsAreOff.java
java -ea modules/21-testing-your-code/examples/AssertionsAreOff.java

java -ea modules/21-testing-your-code/examples/UsingAssertions.java
java modules/21-testing-your-code/examples/MicroHarness.java
java modules/21-testing-your-code/examples/WhatJUnitAdds.java

./scripts/verify-examples.sh modules/21-testing-your-code
```

Run the first one both ways. The difference is the lesson.

Common Mistakes

Assuming `assert` runs. It does not, unless someone passed `-ea`.

Side effects inside an `assert`. `assert list.remove(x)` stops working in production.

Validating a caller's arguments with `assert`. Use an exception.

Tests that depend on each other. The second passes because the first ran, until someone reorders them.

A test with no detail on failure. "FAIL" tells you nothing; the expected and actual values tell you everything.

A harness that dies on the first exception. One bad test should not take down the run.

Writing your own framework for a real project. Everything above is the argument for using JUnit, not against it.

Key Takeaways

- **Assertions are disabled by default**, so `assert` is not a testing mechanism.
- **Never put logic or argument validation in an `assert`.** Internal invariants only.
- **A test harness is a method that checks something and reports.** Thirty-nine lines is enough to prove it.
- **Catch `Throwable` in a harness** so one bad test does not end the run.
- **Failures must carry expected and actual**, or debugging starts from nothing.
- **Use JUnit on real projects.** This module is why it exists, not a substitute.

Homework

[homework/README.md](#)

Extend the harness with the features a framework would give you, then find the bug `assert` hides.

Reference solution in [solutions/21-testing-your-code/](#).